Eliminating Timing Channels in Microkernels

MSc Project Description

Simon Vinding Brodersen(rzn875)

April 22, 2026

1 Description

1.1 Background and Motivation

For decades, the dominant threat model for software security has centered on memory safety vulnerabilities, including buffer overflows, format-string bugs, use after free errors, and integer conversion flaws. These vulnerabilities have historically been the industry's "low hanging fruit" for attackers, because exploitation is straightforward and consequences are severe. Modern cyberattacks still rely heavily on these vulnerabilities, which remain in systems software written in C and C++.

In response, the industry is increasingly adopting strong countermeasures, including memory-safe languages like Rust, sophisticated compiler mitigations (e.g., stack canaries), and dynamic analysis tools. As these defenses mature and memory exploits become harder to execute, attackers are shifting their attention to "higher-hanging fruit" including but not limited to: timing channels, cache-based side channels and interrupt-based leaks all part of the term "side-channel attacks".

Unlike memory corruption, which alters the program's state or control flow, timing attacks exploit the physical execution of the software. Timing channels arise when execution time depends on secret or security-relevant internal states. An attacker can observe execution time, and infer sensitive information such as private keys, memory-access patterns, scheduling decisions, or resource contention. Timing channels violate intended isolation boundaries at the process, VM, or network level, allowing seemingly independent programs to leak information indirectly. Such attacks have already been shown to be detrimental and as such it is not something that can be left untouched [1], [2].

A variety of countermeasures have been proposed to address these issues. In cryptography domain specific languages such as FaCT provide transformations of potentially timing sensitive high level code to low level constant time code [3]. This ensures that in an isolated environment the low level code will not leak information about its secrets through timing side channels. However, practical systems rarely operate in such controlled environments: operating systems provide no guarantees regarding scheduling decisions and do not flush caches or registers on context switches. Consequently, opportunities for microarchitectural channel leaks can still occur.

Previous work has been done to attempt to prevent such leaks. For instance hardware specific extensions have been implemented which provide mechanisms for clearing vulnerable micro architectural state and guaranteeing a history independent context switch latency [4], [5]. Other work has examined the work required for time protection and have provided an implementation for the seL4 microkernel showing it is possible to protect against with little impedance on performance [6].

However, evaluating timing-channel resistance remains difficult. Existing approaches often rely on empirical measurements on real hardware, specialized platforms such as FPGAs, or manual inspection of execution traces. There is currently no widely available, systematic platform for analyzing and reasoning about cache-induced timing channels in a controlled and reproducible manner.

This thesis addresses this gap by focusing on the cache layer as a primary source of timing leakage and by providing an emulation-based framework for analyzing and mitigating such channels.

1.2 Problem statement

Despite their importance, timing side channels, particularly cache based channels, remain insufficiently defended in mainstream systems. While constant time programming techniques can eliminate data dependent control flow at the instruction level, they do not prevent leakage caused by shared cache state, context switching, or operating system scheduling decisions.

The central problem this thesis addresses is: *To what extent can timing side-channels be mitigated through a combination of hardware aware software design and what hardware support is required?*

Specifically, this project will provide a plugin integration to the QEMU virtual machine. This implementation models cache behavior for configurable cache architectures and enables detection of potential timing channels arising from cache contention, eviction patterns, and shared microarchitectural state. The goal is to provide a deterministic and reproducible environment in which timing-channel behavior can be

studied without relying solely on empirical hardware measurements.

This implementation will be tested using previous ”timing” safe microkernels, such as the [7], to show that it is effective in show cache based timing channels. Further, this will then be used as the development basis to implement timing protection in a non-timing safe kernel. Specifically this will be done by looking into deterministic context switching, targeted flushing of the microarchitectural state and constant-time system calls. These implementations will then be evaluated in regards to the possible cache timing channels using the aforementioned plugin and showcase whether it is feasible to implement timing protection in a more standard kernel.

This project further wants to establish hardware requirements for such mitigations to work efficiently and what division of responsibility between hardware and software is required to enable practical timing-secure execution in microkernel based systems.

1.3 Project-Specific Learning Objectives

- Explain and understand the theoretical and practical mechanisms of side-channel leaks across hardware, OS and program layers.
- Survey existing side-channel mitigation techniques including but not limited to: hardware mechanisms and OS-level strategies.
- Design and implement a cache emulation layer capable of finding timing channels in the cache layer.
- Evaluate the efficacy and viability of the implementation by running already established microkernels.
- Modify existing microkernel to provide better timing channel mitigations and evaluate the efficacy along with performance hindrance.
- Gather the findings into design recommendations for future operating systems, hardware abstractions or tool chains, showing how side channel-secure execution could be supported in the future.

Contents

1 Description **1**
1.1 Background and Motivation 1
1.2 Problem statement 2
1.3 Project-Specific Learning Objectives 3

2 Introduction **4**

3 Background **5**
3.1 Side-channels 5
3.2 Covert channels 7
3.3 Isolation 7
3.4 Time protection 8
3.5 Cache coloring 8

4 Related Work **8**
4.1 The missing OS Abstraction: sel4 8
4.1.1 Threat model 8
4.1.2 Channels 8
4.1.3 Cloning the OS 9
4.1.4 Cache coloring 9
4.2 temporal_fence_t 10
4.2.1 With software support 11
4.3 S3K 11
4.3.1 Implementation 11
4.3.2 Time protection 12
4.3.3 Evaluation 13
4.4 FreeRTOS 13

5 Scheduling **14**
5.1 The FreeRTOS scheduler 14
5.2 Closing Scheduling Based Timing Channels 15
5.3 Implementation 17
5.3.1 Extending the api 17
5.3.2 New configuration flags 19
5.3.3 Domain switching 20
5.3.4 Creating domains 22
5.3.5 Per domain scheduling 25
5.3.6 Cross domain communication 26

2 Introduction

Side-channels occur when a program is able to gather information through channels not meant for communication. One such example is timing channels, which occur

when a programs execution time varies based on information throughout the program execution. This way timing channels bypass the operating system's (OS's) security enforcement leaking information through the timing of observable events. Prominent examples of such channels includes side channels in cryptographic libraries [8], [9]. Further, attacks showed that it is possible to do across VM's running in a cloud computing setting [10]. Not only that, side-effects from out-of-order executions on modern processors gave way for the meltdown attacks to read arbitrary kernel-memory locations including personal data and passwords [2]. Mitigations against the Meldown attacks, where kernel memory is hidden from user space are moving forward [11] Mitigations against these attacks have been proposed. Notably in cryptography a push towards more constant-time libraries gave way to implementations such as FaCT [3], which aims to guarantee constant-time execution with respect to information marked secret. However, while this does produce seemingly constant-time program, CPU's speculative execution still leaves doors open for timing attacks such as the ones seen in [1].

The attack front is clear, but the solution is not as simple. While some proposed solutions focus on the user side implementing secure programs, this creates a huge burden on the programmer to implement secure code. Instead implementations of secure microkernels [6], [7] attempt to guarantee security through the OS. The operating system would then be in charge of partitioning domains in regards to the microarchitectural state, such that no channel between domains is possible. This is already done for memory, and examples of separation kernels on the memory level have been formally proven [12], similar work attempts to provide same rigorous proof for the absence of timing channels [13]. However, currently no formal proof for multicore systems exists, and the proof also relies on the secure system to track the touched addresses.

3 Background

3.1 Side-channels

Side-channels are unintended information channels, that leak information about the process. These channels infer secret data often through observable execution time of the process. The general case of such a channel can be seen in the snippet below:

This algorithm is a naive example, where we simply branch on some secret, and the execution time of the program. If for example this algorithm was part of some library, then an attacker could infer information about the secret by simply measuring the execution time of the call to the algorithm. This example is quite naive, and is

```
if secret then
    Some long computation
else
    Do nothing
end if
```

one of the side channels constant time programming can stop, such that we might do an equally long computation in both of the branches of the if statement. However, this still comes with some precautions, because even though the original program might seem sensibly constant time, some compilers might try to optimise the code, which could introduce new side-channel attacks [14].

Further, more complex examples of side channels includes those specified within [1], where speculative execution can be used to leak information about an otherwise safe algorithm. An example from the paper is the following:

```
if x < array1_size then
    y = array2[array1[x] * 4096]
end if
```

This function could again be part of some library of system call, which receives from the user an unsigned integer x. The process, which runs this library code or system call has access to both array1 and array2. It does a bounds check on x, to make sure we have no out of bounds access, which could trigger an exception or access sensitive memory. If this algorithm was called multiple times, with correct input where $x < array1_size$, then we might train the branch predictor on modern CPUS such that it would speculatively execute the "safe" path, which could bring in the location read from `array2` into the cache, and the attacker could then use attacks such as Flush+Reload or Prime+Probe to reveal which part of array2 was loaded into the cache. With this, it can gather the index of the loaded item, and can figure out what was read from array1[x], which could be some arbitrary secret that was obtained by the out of bounds indexing. A more detailed explanation along with a proof-of-concept is provided in [1], but this small example shows how seemingly safe code, might still leak secret information.

A multitude of possible attacks making use of these side-channels are presented in [15], [16].

3.2 Covert channels

While side-channels are passive and exploit unintended leakage, covert channels involve two processes working together. The usually revolves around a sender (Trojan) and a receiver (spy), where the sender has access to high security data but is restricted from communicating directly to the receiver. To bypass this, the sender makes use of some shared resource to signal information to the receiver.

Generally two main categories of covert channels exists; **Storage channels:** which modify some stored object, e.g. a file or some metadata; **Timing channels:** which change the time of different observable events for the receiver. For this thesis storage channels are not referenced, as kernels such as sel4 have already proven the absence of them [12].

Covert timing channels work through the same mechanisms as the aforementioned side channel, but pose as a worst case, where the sender can make use of any and all leakage methods to convey the information. This way, we are not limited to the visible gadgets that might exist in a vulnerable program, but the information might also be conveyed through repeated use of non-constant time system calls, the receiver could then measure the cache misses on the cache sets which is know to be touched by the system calls such as done in the evaluation of [6].

3.3 Isolation

Throughout many timing channel mitigations isolation is a common factor. There exists two types of isolation, namely spatial isolation and temporal isolation. Spatial isolation partitions software into distinct memory spaces, which prevents separate domains from interfering with each other. This separation is most commonly used for DRAM memory, where the OS kernel rely on hardware like the memory management units (MMUs) or memory protection units (MPUs). The MMU translates virtual memory addresses into physical addresses in main memory. It also generally provides memory protection by implementing access control rules and regulations stopping illegal usage of particular memory locations. Another notable examples of spatial isolation is cache coloring, which I will come back to later.

The latter isolation type, namely temporal isolation, must ensure that the service received from shared resources by the software in one domain cannot be affected by the software in another domain [17]. An example, which would require such isolation is any shared part of the OS between different domains. Calling shared OS code should not have any measurable timing affect for another domain calling the same OS code.

To achieve temporal isolation one must clean any microarchitectural state affect by

shared calls. This is where an instruction such as `fence.t` [4] would clear this state. That is, when spatial isolation is not possible, then one must implement temporal isolation by flushing any dirty state before switching domains to ensure complete isolation.

3.4 Time protection

3.5 Cache coloring

4 Related Work

4.1 The missing OS Abstraction: sel4

In the paper Time Protection: The missing OS Abstraction [6] a customized version of the sel4 microkernel is presented, which aims to protect against all possible timing channels via. the OS.

4.1.1 Threat model

In the paper they present the threat model as being a victim program, where a Trojan within the program uses any means necessary to attempt to leak information through timing channels. There are no restrictions on the Trojan outside it simply running in a separate domain than that of the receiver/spy program.

Specifically they differentiate between two types of channels, namely the **covert** and **side** channel. The covert channel requires the collusion between the sender and the receiver and represents the worst-case scenario, where the sender intentionally attempts to transmit information. The side channel is instead when the sender leaks information to the receiver unwittingly. That is, the sender leaks information via normal program execution. By protecting against the covert channel it immediately follows, that the side channel is protected.

4.1.2 Channels

Two main types of channels are referenced throughout the paper. The on-core state, including the L1, L2 caches, Translation Lookaside Buffers and more, which are unique to each core on the CPU and the shared state which includes higher level caches that are shared between different cores. To protect against all, they provide the following five core requirements:

- *Flush on-core state:* When time-sharing a core, all on-core microarchitectural state must be flushed on domain switch, unless the hardware supports specific partitioning of this state.
- *Partition the OS:* Each security domain must have a private copy of OS text, stack and global data. That is, all the dynamic OS kernel memory must have the same separation as userland execution.
- *Deterministic data sharing:* Access to any remaining shared OS data must be sufficiently deterministic to avoid information leakage.
- *Flush deterministically:* State flushing must be padded to its worst case latency.
- *Partition interrupts:* When sharing a core, the OS must disable or partition any interrupts other than the preemption timer. (This is only a concern intra-core).

4.1.3 Cloning the OS

In the paper they introduce a method for cloning the OS. A userland program, which has the capability of cloning gives up a subset of it's own allocated memory for the clone call, and the kernel then creates a kernel image copy in the user-supplied memory. This memory now gets marked as kernel data, meaning that the userland can not change it, but that any syscalls will run through this kernel copy.

This works via the security domains of sel4. The initial kernel creates a master Kernel_Image which represents the present kernel and includes the clone right. It hands this capability along with the size of the image to the initial user process, which then partitions the memory into security domains, where it clones the OS into each of these new security domains.

This cloning consists of three steps:

1. The user process retypes some Untyped memory into an uninitialized Kernel_-Image and Kernel_Memory of sufficient size.
2. it allocates address space identifier to the uninitialised Kernel_Image.
3. it invokes the Kernel_Clone on the new Kernel_Image.

4.1.4 Cache coloring

To protect the caches that are too large to be flushed, cache coloring is implemented. This means, that separate security domains are unable to access the same parts of

caches, such that they have no influence on each other. This reduces the total amount of available cache space for each domain, and it means that the process which creates each security domain has to take this factor into account, such that the distribution of the cache space is sufficient for all domains.

4.2 temporal_fence_t

Following the difficulties found in [6], it was clear that hardware needs improvements before time protection truly could be achieved. In the paper [4] they implement such a hardware improvement. First, it is demonstrated that there currently exists presence of serious timing channels within the in-order RISC-V core and confirm that the current ISA lacks mechanisms to close them reliably.

The initially present two version of the `fence.t` instruction, a basic flush which exclusively flushes the L1 data and instruction caches, the TLBs, and the branch predictors. Through their evaluation they find that further stateful components such as the LFSR providing pseudo-random index sequence for the cache replacement policy and the round-robin memory arbiters of the core still leak data in the microarchitectural state. As such, they also implement a Full Flush, which clears these components along with the once in the basic flush. To use this full flush efficiently the `fence.t` proceeds as follows.

- Save the program counter.
- Save locally modified (dirty) architectural state.
- Drain pending transactions.
- Clear components that are not cleared on reset.
- Assert Microreset. Clearning all flip-flops containing non-architectural state.

A full explanation of each step is provided in [4].

Given this implementation, they augment the sel4 kernel to always call the `fence.t` instruction whenever there is a context switch, and they find that the overhead is around 0.7%. It is noted, that this should be seen as a pessimistic result due to the use of idle tasks, and that in a setting with high memory contention, the latency may be almost entirely hidden.

4.2.1 With software support

While `fence.t` was effective and inexpensive on simple in-order cores. However, in a follow up paper [5] argue that in out-of-order execution CPUs there are challenges with the implementation and address these challenges by presenting a SW-supported temporal fence `fence.t.s` methodology. They show that this software supported version is able to close all on-core timing channels without measurable hardware overhead at a minimal performance impact of 1.0% on average.

4.3 S3K

S3k provides an implementation for a real time operating system, which aims to provide a capability-based multicore partitioning kernel for embedded RISC-V systems. S3K provides spatial and temporal isolation[1], time protection and dynamic resource reconfiguration. Further, S3K's scheduler guarantees deterministic process dispatch, free from microarchitectural interference. [7]

It does via. 3 main attributes

- Capability model;
- Time-driven capability based scheduler with systematic use of `fence.t` with worst time padding, and constant time kernel operations;
- Predictable constant time system calls.

4.3.1 Implementation

The implementation provides a minimal trusted computing base (TCB) with strong isolation and security guarantees. The basic execution unit for S3K is a *process*, a self contained entity with resources defined by its capabilities. These processes are designed to host bare-metal real-time operating systems or applications, with the kernel acting as a bare-metal hypervisor. The number of processes is statically defined at compile time. Processes use *monitor capabilities* and capability revocation to manage other processes, reclaim resourceses, and repurpose existing processes for new tasks. [7]

[1]Spatial means separation of memory, where as temporal indicates that the failure of one component should not affect the schedulability of another.

Capabilities: Every S3K capability operation has a bounded worst-case execution time (WCET) and is free of timing side-channels. Capabilities are implemented in software, and are stored in a capability derivation tree (CDT), which are stored in memory in a statically allocated capability table (ctable).

Scheduling: S3K's scheduler is a partition scheduler, which divides time into *major frames*, which are further partitioned into *minor frames*. S3K treats minor frames as first-class capabilities, which enables dynamic reconfiguration of computing resources while maintaining process isolation. [7]

A major frame is divided into a fixed number of time slots, each with equal duration. A single time slot represents the smallest scheduling unit. A time slice capability grants control over a range of time slots on a hardware thread(hart). The process that owns the time slice capability receives a minor frame, which is less than or equal to the major frame. Each time slice capability has an *enable* field, which specifies if the minor frame is active. If set, the owner of the time slice is scheduled during the corresponding minor frame; otherwise the minor frame represents idle time.

The kernel is in principle non-preemptable, it uses preemption points, to ensure the WCET is bounded by constant. The kernel checks for preemption at the start of every system call, and aborts the system call if preemption is required.

By the definition of domains in S3K processes from different domains are never scheduled in parallel. Thus, during a domain switch, all running processes of the current domain are preempted, invoke the scheduler, and are subsequently delayed by the temporal fence.

4.3.2 Time protection

S3K uses the temporal fence instruction at context switches, which flushes the core-local microarchitectural state.

S3K is designed to reside entirely within the small scratchpad memory (SPM) backed by the core-local L1 cache.This enables the kernel's microarchitectural footprint to be efficiently flushed and avoids reliance on larger caches.

S3K only protects a process's in-kernel state from side-channels. It does not protect a process's user-level state. Each process is responsible for protecting their own user-level execution from side-channels in higher level caches. This is different than what is provided within [6], where cache-coloring guarantees that processes in different domains do not access the same parts of the cache, and thus prevents side channels even in larger cases.

All system calls are designed to be non-interfering and constant-time with respect to the invoking process's authorized observations; that is, all control flow decisions and memory accesses depend only on data the process is permitted to observe.

4.3.3 Evaluation

They evaluate the implementation by looking at a variety of different elements, most significant for ours is: scheduling jitter and side-channels. Initially they find that the Inter Process Communication (IPC) operations have the highest recorded non-preemptable cost of 1,195 cpu cycles, together with an estimated 16,000 cycles for data cache write-back for temporal fence, they set the CSPAD to 18,000 cycles for the fence.t instruction.

Scheduling Jitter: A measurer process, shceduled once per major frame, records the *mcycle* performance counter value upon dispatch. To simulate interference they introduce interfering processes that execute random system calls and pollute the cache. They find, that when the measurer exclusively occupies the SPM, there is no impact from the other processes on the measurements. However, when the measurer occupies the dram, then there is noticeable jitter, which they note is in some cases due to the small part of code in the measurer that records the dispatch time and not the scheduler itself. When more than one process is scheduled at a time, then there is jitter in the scheduler as well.

Side channels: They implement a Trojan process and a spy process. The Trojans signal a "0" by remaining idle and a "1" by behaving like the interferes from before. The spy then measures jitter, system call execution times, and cache access times to infer the Trojans' signal.

They find that when the measurer and Trojan both reside in the dram, then there is a near perfect information flow, whereas when both reside in the SPM, there is no measurable amount of information flow.

4.4 FreeRTOS

FreeRTOS is a class of Real Time Operating Systems(RTOS), that is designed to be small enough for microcontrollers while not limiting its application to purely microcontrollers. Out of the box, FreeRTOS provides core real time scheduling functionality, inter-task communication, timing and synchronisation primitives along with other possible add on functionalities. [18]

FreeRTOS-MPU is a memory protected version of FreeRTOS, which as of writing is supported on select ARM based microcontrollers. It enables tasks to run in either privileged or unprivileged mode and restricts access to resources such as RAM, executable code, peripherals and memory beyond the limit of a task's stack. [19]

Currently there exists a third party port of this implementation to the RISC-V ISA [20]. This is what forms the basis for this thesis implementation. There are a couple of reason for this choice. The first and most important one is familiarity. Previously I have worked with the RISC-V ISA and as such this reduces the overhead needed on my end in regards to working with a different ISA. Second, the related work on timing channels generally focuses on this ISA, so future comparison between implementations are more relevant if all are built on the same platform. At last, the `fence.t` instruction introduced in Section 4.2 is implemented with the RISC-V instruction set in mind. While the underlying idea could be implemented on other platforms work on hardware support is not as far along, so assuming this implementation will run on real hardware at some point in the future, then RISC-V seems like the most reasonable choice.

5 Scheduling

5.1 The FreeRTOS scheduler

FreeRTOS provides different scheduling algorithms based on configurations and number of cores available on the system. By default on a single core FreeRTOS makes use of a fixed-priority preemptive scheduling policy, with round-robin time-slicing of equal priority tasks. [21]

As such, FreeRTOS always attempts to schedule the highest priority task and possibly starves lower priority tasks. Items of the same priority run until a tick interrupt at which point the time slice is up. This type of scheduling inherently includes timing side-channels as the higher priority tasks inhibit lower priority tasks from running. [22], [23] This could be used as a covert channel, where the higher priority task could signal either a 0 or 1 by simply going to sleep and the lower priority task measures its own execution time.

Therefore, for FreeRTOS to provide timing protection the scheduler must see modifications.

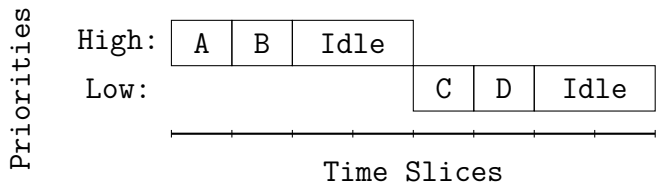


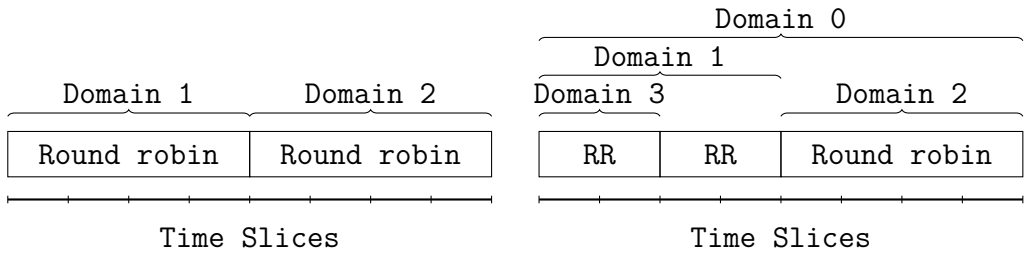
Figure 1: Time protection within same priority

5.2 Closing Scheduling Based Timing Channels

There are a few approaches to closing the different side channels each with their own pros and cons. A general attribute among all of them is, that starvation must not be possible, and as such the current approach to priority based scheduling is infeasible.

Removing time: One approach to solve a lot of the timing channels would be to disallow tasks to read the current time. This in theory would remove the scheduling based timing channel, but this solution would most likely attributed quite a significant decrease in the possibilities of tasks, as any action, which could leak time information to the tasks would need to be prohibited. As such, this approach was not considered further.

Protection within priority: If the goal is to keep a sense of priority based scheduling, then it should be possible to guarantee timing protection for tasks of same priority or tasks of different priorities. Say for example, we for each priority have a known array which constitutes the time slices for the given priority. When the scheduler attempts to schedule a task, then it prioritizes the higher priority array if a task is ready within, and uses time slice based scheduling within the priority. An example of this can be seen visualized in Figure 1, where tasks A and B are within the same priority. Here, if we assume that A and B are runnable, then they would be scheduled before the lower priority C and D. This would allow C and D to gather information about the run ability of the entire High priority noting that either A or B was runnable if C or D are delayed. However, within the same domain tasks A and B can not gather information about each other. Say for example A gets to run, then it does not know whether task B is runnable or not, as we are always running the entire time slice given defined by the aforementioned array.



(a) Static domain scheduler (b) Dynamic domain scheduler

Figure 2: Domain scheduling

This approach would by no means close all timing channels, as mentioned cross priorities could still leak timing information. Other constraints with this approach is, that we are limiting the number of possible tasks for given priorities. In the previous example 4 time slices were provided within the High and Low priorities, and we would not be able to fill more than these 4 time slices as we would then change the total time slice when running a specific priority and thus a task within the given priority could gather information about the total number of time slices. Another issue would be, that we can't simply give up CPU time. When a given tasks wishes to sleep for a short duration, we must replace the time slice with idle time, which significantly reduces overall CPU utilization. Once a task is completely finished it is possible to then take back the time slice, but the new task that would be scheduled in this slot would then gather the information that the previous task finished its execution.

Domain scheduling: Another approach is the one seen in [6], which is the domain level scheduler. The general idea is quite similar to the approach seen before, but the idea of priorities have are removed entirely and instead we focus on domains. Each domain can have multiple tasks within, where we do not care about timing channels and only between different domains do we care about timing protection. In Figure 2a an illustration of this scheduler can be seen. The idea is, that at compile time, we know the different domains and attribute to each some amount of time slices (in the example that would be 4 for both domain 1 and 2). The domain scheduler then takes care of scheduling the domain, and guarantees timing protection only between the different domains. This approach is still inhibited by the need to know the total number of time slices at compile time, and further it also needs the number

of time slices for each domain at compile time. However, where this benefits over the previous approach is, that within each domain there is a lot more freedom to do as pleased. For example, one could imagine having a regular round robin scheduler within a domain, which then allows more CPU utilization within this specific domain and less unused resources.

An extension of this approach would be the on seen in Figure 2b, which closely follows the capability based scheduler in [7]. The idea is, that within a specific domain, we can give up some of the attributed time to create a new subdomain. This would mean that there is timing protection between subdomain 1 and the rest of domain 1[2]. In theory it should be possible to give up these time slices back to the domain level scheduler, which would then treat this as a new domain. Whether or not some timing channels persists after creation remains to be seen.

This approach seems by far the most flexible and most robust in terms of real world applications. As such, this thesis attempts to implement this domain scheduler within the FreeRTOS kernel.

It is important to note, that this approach does not remove all timing channels. Each domain uses a round robin scheduler, which as mentioned previously is susceptible to timing channels. The idea is instead, that we allow such channels in tasks where it is not relevant for protection and throughput is valued higher. Instead, when we explicitly want a safe execution environment we can create a domain for this, which is then protected after initialization.

5.3 Implementation

5.3.1 Extending the api

To implement the domain level scheduler we have first incorporate the idea into the existing FreeRTOS framework. With FreeRTOS MPU, we are able to create unprivileged tasks which have restricted access to syscalls and provide them with a memory space that is partitioned. For this, the general FreeRTOS api is extended with a multitude of new functions most importantly for this project is the `xTaskCreateRestricted`.[19] The idea behind the domain scheduling fits with the purpose of memory protection, and as such it seems the obvious choice for modification to extend it with the domain functionality. The signature of the aforementioned function is provided in the listing below:

[2]Other than creation and finishing assuming a domain is allowed to give back the time slices upon finishing, which would leak timing information

```c
typedef struct xMEMORY_REGION
{
    void * pvBaseAddress;
    uint32_t ulLengthInBytes;
    uint32_t ulParameters;
} MemoryRegion_t;

typedef struct xTASK_PARAMETERS
{
    TaskFunction_t pvTaskCode;
    const char * pcName;
    configSTACK_DEPTH_TYPE usStackDepth;
    void * pvParameters;
    UBaseType_t uxPriority;
    StackType_t * puxStackBuffer;
    MemoryRegion_t xRegions[ portNUM_CONFIGURABLE_REGIONS ];
    #if (
      ( portUSING_MPU_WRAPPERS == 1 ) &&
      ( configSUPPORT_STATIC_ALLOCATION == 1 )
    )
        StaticTask_t * const pxTaskBuffer;
    #endif
} TaskParameters_t;

BaseType_t xTaskCreateRestricted(
  const TaskParameters_t * const pxTaskDefinition,
  TaskHandle_t * pxCreatedTask
) PRIVILEGED_FUNCTION;
```

Listing 1: xTaskCreateRestricted function signature

That is, we can create a task which has access to a number of memory regions defined by the region start address (pvBaseAdddress), the region size in bytes (ulLengthInBytes) and the access permissions for the region (ulParameters). The function parameter uxPriority is also of importance. By default this function creates tasks running in user mode, which restricts access to specific system calls and any task created running in privileged mode is able to access the entire RAM. By default, tasks running in user mode are able to create other tasks running in user mode, and this is the desired behaviour for domain scheduling.

The idea is to extend this function signature as follows:

```c
...

typedef struct xDOMAIN_PARAMETERS
```

```
{
  uint32_t ulSliceIndex;
  uint32_t ulSliceLength;
} DomainParameters_t;

typedef struct xTASK_PARAMETERS
{
  ...
    #if ( configENABLED_DOMAINS == 1 )
        DomainParameters_t * xDomainParameters;
    #endif
} TaskParameters_t;

BaseType_t xTaskCreateRestricted(
  const TaskParameters_t * const pxTaskDefinition,
  TaskHandle_t * pxCreatedTask
) PRIVILEGED_FUNCTION;
```

Listing 2: Extended parameters and signature

A task with the capability to create a new restricted task would then have to provide the index in the global time slice array, along with the length of the new domain that will be created. If no xDomainParameters is passed to the function (i.e. the pointer is NULL) this will be treated as creating a task within the same domain, and will be handled by the per domain priority based scheduler.

With this, no more of the external api has to be changed.

5.3.2 New configuration flags

To enable the functionality of domain level scheduling, a few new compile flags needs to be introduces.

configENABLE_DOMAINS A binary flag to indicate whether or not to use the domain level scheduler alongside the default priority based scheduler.

configNUM_TIME_SLICES The total number of time slices for the domain scheduler to take into account. As mentioned before this flag also constitutes the maximum number of possible domains, such that any domain scheduling decisions can be done in constant time.

connfigNUM_TICKS_PER_SLICE The number of timer interrupt ticks to constitute a single time slice. Say that the ticks is defined as `#define configTICK_RATE_HZ ((TickType_t) 1000)` with `#define configNUM_TICKS_PER_SLICE (5)`, this would mean that a single time slice is equal to TIME_SLICE $= \frac{1s}{1000} \cdot 5 = 5ms$.

5.3.3 Domain switching

To keep with the fundamentals of FreeRTOS the idea is to keep the functionality of the priority based scheduler within FreeRTOS but internally in a domain. That is, within a specific domain the same priority based scheduler is available. This in turn means that if no new domains are created, then the functionality of the scheduler should be nearly unchanged[3].

FreeRTOS makes use of a single static file `tasks.c` for the development of the priority based scheduler. In this thesis I am only focused on the single core implementation and while work on the multicore version would be interesting, that is left for future work.

In the MPU version of FreeRTOS when a machine timer interrupt occurs it is handled in the following manner:

```
freertos_risc_v_mtimer_interrupt_handler:
    portcontextSAVE_INTERRUPT_CONTEXT
    portUPDATE_MTIMER_COMPARE_REGISTER
    call xTaskIncrementTick
    beqz a0, exit_without_context_switch
    call vTaskSwitchContext
```

Listing 3: FreeRTOS mtimer interrupt handler

It increments a global tick, and switches the task content. However, FreeRTOS only increments the global `xTickCount` if the priority based scheduler is running and not suspended for critical handling. This means, we have to introduce our own proxy for Ticks, as the per domain `xTickCount` would leak information about other scheduler states and possibly delay the domain switching. For our purposes we can hook into the pre-existing `xTaskIncrementTick` function, but add tracking for our own tick. as check for a domain switch.

```
/* Static variable to contain the current Domain Tick */

PRIVILEGED_DATA static size_t pxCurrentDomainIndex = 0;
```

[3]There will be some overhead with the implementation of course.

```
PRIVILEGED_DATA static volatile TickType_t xEffectiveTick = ( TickType_t ) configINITIAL_TICK_COUNT;
PRIVILEGED_DATA static volatile TickType_t xDomainTick = ( TickType_t ) 0;

BaseType_t xTaskIncrementTick( void ) {
...
const TickType_t xConstTickCount = xEffectiveTick + ( TickType_t ) 1;

/* Increment the effective tick. This is different than xTickCount,
 * which does not increment if the scheduler is in a critical section.
 * The Domain scheduler does not care about critical section */
xEffectiveTick  = xConstTickCount;

/* Update the Domain tick in regards to the effective tick */
xDomainTick = (xEffectiveTick / configNUM_TICKS_PER_SLICE) % configNUM_TIME_SLICES;

/* Time slot expired or wraparound */
if( xDomainTick >= pxCurrentDomainIndex + xDomains[pxCurrentDomainIndex].uxLength
    || xDomainTick < pxCurrentDomainIndex )
{
    /* Save current task to logical domain info */
    xDomainInfo[xDomains[pxCurrentDomainIndex].uxDomainID].pxPreviousTCB = pxCurrentTCB;

    xSwitchRequired = pdTRUE;

    /* Flush on domain switch */
    temporal_fence_t();

    /* Contiguous partition: next block starts exactly here */
    pxCurrentDomainIndex += xDomains[pxCurrentDomainIndex].uxLength;
    if( pxCurrentDomainIndex >= configNUM_TIME_SLICES )
    {
        pxCurrentDomainIndex = 0;
    }

    /* Restore previous task from logical domain info */
    TCB_t * previous_tcb = \
      xDomainInfo[xDomains[pxCurrentDomainIndex].uxDomainID].pxPreviousTCB;
    /* First time, use idle task */
    if (previous_tcb == NULL) {
        pxCurrentTCB = xIdleTaskHandles[ 0 ];
    } else {
        pxCurrentTCB = previous_tcb;
```

```
    }
}
...
}
```

Listing 4: Custom hook for xTaskIncrementTick

As such, whenever there is a machine timer tick, we calculate in constant time the time slice (xDomainTick) and check if we should do a domain switch. If we find that we should schedule a new domain, then we switch the data to the new domain and perform a temporal flush to clear the microarchitectural state. The flush happens in such a manner, that any data accessed in regards to the new domain is non-interfering with the previous domain. Once we have decided that a switch should occur, only data that is within the incoming domain is retrieved following the flush, this means that the new domains execution will not be based on the microarchitectural state of the previous domain.

Note this comes with some constraints. Most notably we are defining a time slice in relation to the machine timer interrupt timing, which is configurable in FreeRTOS through the `configTICK_RATE_HZ`. To make this more flexible, the `configNUM_TICKS_PER_SLICE` as to not limit the domain level scheduler with respect to the inter domain priority based scheduler.

5.3.4 Creating domains

To keep track of the different domains new global constants are defined:

```
typedef struct DomainBlock {
  UBaseType_t uxDomainID; /* This time slices domain ID */
  size_t uxLength; /* Length of this domain */
} DomainBlock_t;

/* Timeline over configNUM_TIME_SLICES, indexed by the start of a block */
PRIVILEGED_DATA static DomainBlock_t xDomains[configNUM_TIME_SLICES] = {
    [0] = {.uxDomainID = 0, .uxLength = configNUM_TIME_SLICES}
};

typedef struct DomainListItem {
    size_t uxBlockStart;
    struct DomainListItem* next;
} DomainListItem_t;

PRIVILEGED_DATA static DomainListItem_t
```

```
    ↪ xDomainItemPool[configNUM_TIME_SLICES] = {
    [0] = {.uxBlockStart = 0, .next = NULL}
};

typedef struct DomainInfo {
    TCB_t * volatile pxPreviousTCB;
    DomainListItem_t * pxFirstBlock;  /* Singly-linked list of this
    ↪ domain's owned blocks */
} DomainInfo_t;

/* Timeline over Domain information indexed by the Domain ID */
PRIVILEGED_DATA static DomainInfo_t xDomainInfo[configNUM_TIME_SLICES] = {
    [0] = {.pxPreviousTCB = NULL, .pxFirstBlock = &xDomainItemPool[0]}
};

static DomainListItem_t * pxAllocateBlockItem( void )
{
    if( uxNextFreeDomainItem >= configNUM_TIME_SLICES )
    {
        return NULL;
    }
    return &( xDomainItemPool[ uxNextFreeDomainItem++ ] );
}
```

Listing 5: Domains definitions

The `xDomains` array contains the timeline of the Domains. It contains a single entry for every time slice that is possible. This allows for constant indexing as it has a fixed size. One might be able to introduce a vector based approach instead, but this would mean that allocation of a new domain would leak a small amount of information about the total number of domains when the vector has to expand, this is something that for now was not considered further.

The `xDomainPool` contains a pool of compile time allocated `DomainListItem_t`.

The `xDomainInfo` array is indexable by the DomainID, and contains information about the previously running task for restoration upon a reentry of the domain, and a single linked list of owned blocks of time slices by the current domain. This information is used to check at domain creation if the current domain owns the time slice it is attempting to allocate. Further, it is created in this manner, such that the creation of a domain entirely depends on the owned time slices of the domain itself, and does not leak information about the total number of domains within the system, which would be leaked, if we simply loop over the `xDomains` array using the lengths. So while information about the total number of domains owned by this

current domain could be leaked by attempting to create a new domain, other domains information should not be leaked.

With this in place, it allows the creation of new domains as follows:

```
static BaseType_t prvCreateDomain(
    DomainParameters_t * pxParams,
    UBaseType_t * puxNewDomainSlot)
{
    size_t start = pxParams->ulSliceIndex;
    size_t length = pxParams->ulSliceLength;

    if (length == 0)
    {
        return pdFAIL;
    }

    taskENTER_CRITICAL();

    UBaseType_t uxCurrentDomainID =
        xDomains[pxCurrentDomainIndex].uxDomainID;

    DomainInfo_t * pxInfo =
        &xDomainInfo[uxCurrentDomainID];

    DomainListItem_t * pxBlock =
        prvFindContainingBlock(pxInfo, start, length);

    if (pxBlock == NULL)
    {
        taskEXIT_CRITICAL();
        return pdFAIL;
    }

    size_t blockStart = pxBlock->uxBlockStart;
    size_t blockEnd =
        blockStart + xDomains[blockStart].uxLength;

    /* reject overlap with current tick */
    if (start <= xDomainTick &&
        xDomainTick < start + length)
    {
        taskEXIT_CRITICAL();
        return pdFAIL;
    }

```

```
    DomainListItem_t * pxChild = NULL;

    if (prvSplitBlock( pxBlock,
            blockStart,
            blockEnd,
            start,
            length,
            uxCurrentDomainID,
            &pxChild) != pdTRUE)
    {
        taskEXIT_CRITICAL();
        return pdFAIL;
    }

    UBaseType_t uxChildDomainID = uxNextDomainID++;

    xDomains[start].uxDomainID = uxChildDomainID;
    xDomains[start].uxLength = length;

    xDomainInfo[uxChildDomainID].pxPreviousTCB = NULL;
    xDomainInfo[uxChildDomainID].pxFirstBlock = pxChild;

    *puxNewDomainSlot = start;

    taskEXIT_CRITICAL();
    return pdTRUE;
}
```

Listing 6: Function for creating domains

First, we find the containing block of the given new domain, which is to be created. Then, we make sure it does not overwrite the current domain tick, as this could cause issues with the current task. We handle splitting the block if needed via. the helper prvSplitBlock, and at last we update the necessary global items.

5.3.5 Per domain scheduling

For the domain level scheduler to work with inter domain priority based schedulers, we need to add another dimension to all of the items. That is, all variables that previously were a constant would have to be an array with length equal to the number of time slices and same extension for previous arrays. All accesses to these constants/arrays would have to follow a similar transformation, where the currently scheduled domain would choose the correct values for the priority based scheduler to function correctly. A few examples of these transformations are shown below:

```
/* Non-domain scheduler implementation*/
PRIVILEGED_DATA static volatile BaseType_t xSchedulerRunning = pdFALSE;
PRIVILEGED_DATA static volatile TickType_t xPendedTicks = ( TickType_t ) 0U;

/* Domain scheduler equivalent */
PRIVILEGED_DATA static volatile BaseType_t
    ↪ xSchedulerRunning[configNUM_TIME_SLICES] = { pdFALSE };
PRIVILEGED_DATA static volatile TickType_t
    ↪ xPendedTicks[configNUM_TIME_SLICES] = {( TickType_t ) 0U};
```

Listing 7: Transforming the priority based scheduler

Note, that while the first element is initialized here, the initialization of the full items are moved to a separate function, as all of the schedulers would have to be initialized in the same fashion as the default scheduler.

With this, the general foundation of the domain level scheduler is functional, and the standard `mpu_blinky.c` example works given all tasks run within the same domain.

5.3.6 Cross domain communication

As it stands, the FreeRTOS api has next to no information about the different domains, meaning that items such as the Queue used for message transferring between domains is not available. Generally this is a good thing, as by default there is domain isolation, as any internal handling within tasks.c makes use of the current domain. This restriction is quite strict, and one might assume that cross-domain communication should be possible. In FreeRTOS communication happens through Queues. The general workflow for Queue communication in the MPU version of FreeRTOS is as follows:

- Create Queue kernel object in a priority task.
- Grant access to the Queue object to any user level tasks, that should communicate through the Queue.
- One task sends messages through `xQueueSend`.
- Another task receives messages through `xQueueReceive`.

For the cross domain communication the workflow should be the same, but it is the internal handling that has to change. Currently the internal handling of the `xQueueSend` is:

- Check there is room in the queue for the message.
- Check if there is a task waiting to receive on the Queue.
  - If yes, move the task to the ready list.
  - If no, do nothing.

This is generally also fine, except that when moving a task to the ready list, the task might be outside the domain that is currently running. When the Queue moves a task to the ready list, it does so through the function `xTaskRemoveFromEventList`. This function takes a linked list of tasks, namely the tasks waiting on the queue, removes the first tasks from the list and moves the given task into the ready list if the scheduler is suspended and otherwise moves it to the `xPendingReadyList` to wait for the next scheduler suspension before moving into the true ready list. If the new task was found to have a higher priority, then notify that there should be a context switch.

This is where we have to make changes to allow for cross-domain communication. First, each task needs some way of tracking the domain it resides within, which is easily done by extending the `TCB_t` struct with an extra field:

```
typedef struct tskTaskControlBlock
{
    ...
    #if ( configENABLE_DOMAINS == 1 )
        UBaseType_t uxDomainID;
    #endif
} tskTCB;

typedef tskTCB TCB_t;
```

The Queue can now reference this DomainID when modifying the priority-schedulers and as such adds the tasks to the correct arrays.

Given this communication, we can now create the `mpu_blinky.c` example with different domains communicating. It is important to note here, that there of course is introduced some timing leakage between these two domains, as they can time the response time of different messages. Here it is up to the individual domains to respond in a deterministic manner as to not leak any secret information.

References

[1] P. Kocher, J. Horn, A. Fogh, *et al.*, "Spectre attacks: Exploiting speculative execution," in *40th IEEE Symposium on Security and Privacy (S&P'19)*, 2019.

[2] M. Lipp, M. Schwarz, D. Gruss, *et al.*, "Meltdown: Reading kernel memory from user space," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.

[3] S. Cauligi, G. Soeller, B. Johannesmeyer, *et al.*, "FaCT: A DSL for timing-sensitive computation," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2019, New York, NY, USA: Association for Computing Machinery, Jun. 8, 2019, pp. 174–189, ISBN: 978-1-4503-6712-7. DOI: 10.1145/3314221.3314605. [Online]. Available: https://dl.acm.org/doi/10.1145/3314221.3314605 (visited on 02/03/2026).

[4] N. Wistoff, M. Schneider, F. K. Gürkaynak, G. Heiser, and L. Benini, "Systematic prevention of on-core timing channels by full temporal partitioning," *IEEE Transactions on Computers*, vol. 72, no. 5, pp. 1420–1430, May 2023, ISSN: 2326-3814. DOI: 10.1109/tc.2022.3212636. [Online]. Available: http://dx.doi.org/10.1109/TC.2022.3212636.

[5] N. Wistoff, G. Heiser, and L. Benini, "Fence.t.s: Closing Timing Channels in High-Performance Out-of-Order Cores Through ISA-Supported Temporal Partitioning," in *Applications in Electronics Pervading Industry, Environment and Society*, M. Ruo Roch, F. Bellotti, R. Berta, M. Martina, and P. Motto Ros, Eds., Cham: Springer Nature Switzerland, 2025, pp. 269–276, ISBN: 978-3-031-84100-2. DOI: 10.1007/978-3-031-84100-2_32.

[6] Q. Ge, Y. Yarom, T. Chothia, and G. Heiser, "Time Protection: The Missing OS Abstraction," in *Proceedings of the Fourteenth EuroSys Conference 2019*, ser. EuroSys '19, New York, NY, USA: Association for Computing Machinery, Mar. 25, 2019, pp. 1–17, ISBN: 978-1-4503-6281-8. DOI: 10.1145/3302424.3303976. [Online]. Available: https://dl.acm.org/doi/10.1145/3302424.3303976 (visited on 02/03/2026).

[7] H. Karlsson and R. Guanciale, "Partitioning Kernel With Capability Controlled Temporal and Spatial Partitioning," in *2025 IEEE Real-Time Systems Symposium (RTSS)*, Dec. 2025, pp. 68–81. DOI: 10.1109/RTSS66672.2025.00015. [Online]. Available: https://ieeexplore.ieee.org/document/11315078 (visited on 02/04/2026).

[8] J.-F. Dhem, F. Koeune, P.-A. Leroux, P. Mestré, J.-J. Quisquater, and J.-L. Willems, "A Practical Implementation of the Timing Attack," in *Smart Card Research and Applications*, J.-J. Quisquater and B. Schneier, Eds., red. by G. Goos, J. Hartmanis, and J. Van Leeuwen, vol. 1820, Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 167–182, ISBN: 978-3-540-67923-3 978-3-540-44534-0. DOI: 10.1007/10721064_15. [Online]. Available: http://link.springer.com/10.1007/10721064_15 (visited on 02/17/2026).

[9] O. Aciiçmez, W. Schindler, and Ç. K. Koç, "Improving Brumley and Boneh timing attack on unprotected SSL implementations," in *Proceedings of the 12th ACM Conference on Computer and Communications Security*, ser. CCS '05, New York, NY, USA: Association for Computing Machinery, Nov. 7, 2005, pp. 139–146, ISBN: 978-1-59593-226-6. DOI: 10.1145/1102120.1102140. [Online]. Available: https://dl.acm.org/doi/10.1145/1102120.1102140 (visited on 02/17/2026).

[10] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds," in *Proceedings of the 16th ACM Conference on Computer and Communications Security*, Chicago Illinois USA: ACM, Nov. 9, 2009, pp. 199–212, ISBN: 978-1-60558-894-0. DOI: 10.1145/1653662.1653687. [Online]. Available: https://dl.acm.org/doi/10.1145/1653662.1653687 (visited on 02/04/2026).

[11] J. Corbet, "KAISER: Hiding the kernel from user space," *LWN.net*, Nov. 15, 2017. [Online]. Available: https://lwn.net/Articles/738975/ (visited on 02/17/2026).

[12] T. Murray, D. Matichuk, M. Brassil, *et al.*, "seL4: From General Purpose to a Proof of Information Flow Enforcement," in *2013 IEEE Symposium on Security and Privacy*, Berkeley, CA: IEEE, May 2013, pp. 415–429, ISBN: 978-0-7695-4977-4 978-1-4673-6166-8. DOI: 10.1109/SP.2013.35. [Online]. Available: http://ieeexplore.ieee.org/document/6547124/ (visited on 02/17/2026).

[13] S. Buckley, R. Sison, N. Wistoff, *et al.* "Proving the Absence of Microarchitectural Timing Channels." version 1. arXiv: 2310.17046 [cs]. (Oct. 25, 2023), [Online]. Available: http://arxiv.org/abs/2310.17046 (visited on 02/10/2026), pre-published.

[14] M. Schneider, D. Lain, I. Puddu, N. Dutly, and S. Capkun, "Breaking Bad: How Compilers Break Constant-Time Implementations," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, Aug. 25, 2025, pp. 1690–1706. DOI: 10.1145/3708821.3733909. arXiv: 2410.13489

[cs]. [Online]. Available: http://arxiv.org/abs/2410.13489 (visited on 02/17/2026).

[15] D. A. Osvik, A. Shamir, and E. Tromer, "Cache Attacks and Countermeasures: The Case of AES," in *Topics in Cryptology – CT-RSA 2006*, D. Pointcheval, Ed., red. by D. Hutchison, T. Kanade, J. Kittler, *et al.*, vol. 3860, Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 1–20, ISBN: 978-3-540-31033-4 978-3-540-32648-9. DOI: 10.1007/11605805_1. [Online]. Available: http://link.springer.com/10.1007/11605805_1 (visited on 02/10/2026).

[16] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-Level Cache Side-Channel Attacks are Practical," in *2015 IEEE Symposium on Security and Privacy*, May 2015, pp. 605–622. DOI: 10.1109/SP.2015.43. [Online]. Available: https://ieeexplore.ieee.org/document/7163050 (visited on 02/04/2026).

[17] J. Rushby, "Avionics Architectures: Mechanisms, and Assurance," 1999.

[18] "What is FreeRTOS? - FreeRTOS™." (Apr. 2026), [Online]. Available: https://freertos.org/Why-FreeRTOS/What-is-FreeRTOS (visited on 04/07/2026).

[19] "Memory Protection Unit (MPU) Support - FreeRTOS™." (Apr. 2026), [Online]. Available: https://freertos.org/Security/04-FreeRTOS-MPU-memory-protection-unit (visited on 04/07/2026).

[20] "FreeRTOS-Community-Supported-Demos/RISC-V_RV32_MPU_QEMU_-VIRT_GCC at main · FreeRTOS/FreeRTOS-Community-Supported-Demos," GitHub. (), [Online]. Available: https://github.com/FreeRTOS/FreeRTOS-Community-Supported-Demos/tree/main/RISC-V_RV32_MPU_QEMU_VIRT_GCC (visited on 04/07/2026).

[21] "FreeRTOS scheduling (single-core, AMP and SMP) - FreeRTOS™," FreeRTOS scheduling (single-core, AMP and SMP). (Mar. 2026), [Online]. Available: https://freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/04-Task-scheduling (visited on 04/07/2026).

[22] S. Kadloor, N. Kiyavash, and P. Venkitasubramaniam, "Mitigating Timing Side Channel in Shared Schedulers," *IEEE/ACM Transactions on Networking*, vol. 24, no. 3, pp. 1562–1573, Jun. 2016, ISSN: 1558-2566. DOI: 10.1109/TNET.2015.2418194. [Online]. Available: https://ieeexplore.ieee.org/document/7089318/ (visited on 02/09/2026).

[23] C.-Y. Chen, S. Mohan, R. Pellizzoni, R. B. Bobba, and N. Kiyavash, "A Novel Side-Channel in Real-Time Schedulers," in *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, Apr. 2019, pp. 90–102. DOI: 10.1109/RTAS.2019.00016. [Online]. Available: https://ieeexplore.ieee.org/document/8743170/ (visited on 02/09/2026).